CISC 886 – Cloud Computing

Lab 6 Handout: Containerization with Docker

School of Computing, Queen's University, Kingston, Canada

Overview

This lab has two parts. In **Part A**, you will run a Node.js application directly on an EC2 instance and watch it crash due to a runtime version mismatch, the classic *"it works on my machine"* problem. In **Part B**, you will package the same application into a Docker container and see that the problem disappears.

Part A: Runtime Mismatch

1. Launch and connect to an EC2 instance

- AMI: **Ubuntu Server 22.04 LTS**, Instance type: `t3.micro` (or `t2.micro`)
- Security Group inbound rules: **SSH (port 22)** and **Custom TCP (port 3000)**
- SSH in and update the package index:

```
ssh -i your-key.pem ubuntu@<EC2-PUBLIC-IP>
sudo apt-get update -y
```

2. Install nvm and Node 18

`nvm` (Node Version Manager) lets you switch between Node.js versions on the same machine.

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.4/install.sh |
bash
source ~/.bashrc
nvm install 18 && nvm use 18
```

3. Create the application

```
mkdir -p ~/lab6-app && cd ~/lab6-app
```

Create `package.json` and `app.js` using the files provided by the instructor, then install dependencies:

```
npm install
```

4. Run on Node 18: Watch it Crash

```
node app.js
```

> ⚠️ **Warning:** The app will crash immediately with `TypeError: Object.groupBy is not a function`. This is expected. Note the last line of the error: it shows the Node version. That is the clue.

5. Switch to Node 22: It Works

```
nvm install 22 && nvm use 22
node app.js
```

Open a second terminal and test both routes:

```
curl http://localhost:3000/
curl http://localhost:3000/tasks
```

`GET /` returns `mode: local` and the EC2 hostname. `GET /tasks` returns the shopping list grouped by status.

Press **Ctrl+C** to stop the app.

💡 **Note:** Nothing in the code changed. Only the Node version changed. `Object.groupBy` is a JavaScript API only available in Node.js v21 and above. The developer who wrote this app had Node 22 on their laptop, so their code worked. The server had Node 18, so it crashed.

✏️ **Reflect:** What caused the application to crash on Node 18 but work on Node 22? Explain in one or two sentences.

Key Concepts

Term	Meaning
Docker Image	A snapshot of your app, its dependencies, and its runtime, all bundled together. Nothing is running yet.
Docker Container	A running instance of an image. You can start and stop it, and run many from the same image.
Reproducibility	Because the runtime is inside the image, the result is the same on every machine that runs it.

💡 **Note:** Think of an image as a class definition and a container as an object created from it. The image describes the environment; the container is the running instance.

Part B: Containerizing with Docker

6. Install Docker

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
sudo usermod -aG docker $USER
newgrp docker
docker run hello-world
```

> ⚠️ **Warning:** If you see `permission denied` when running a `docker` command, run `newgrp docker` again to activate group membership in your current shell.

7. Write the Dockerfile

In `~/lab6-app`, create a file named `Dockerfile` using the file provided by the instructor. It pins Node 22 as the runtime and bakes `MODE=docker` into the image.

> ✏️ **Reflect:** Where exactly in the Dockerfile is the Node version specified? What would happen if you changed that line to `FROM node:18-alpine` and rebuilt the image?

8. Build the image

```
docker build -t lab6-app .
```

Check that the image was created:

```
docker images
```

9. Run a container

```
docker run -d -p 3000:3000 --name myapp lab6-app
```

Check it is running and view its logs:

```
docker ps
docker logs myapp
```

The logs should show `Mode: docker` and a container ID as the hostname, not the EC2 hostname.

Test both routes:

```
curl http://localhost:3000/
curl http://localhost:3000/tasks
```

💡 **Note:** Compare the output with Part A. The `mode` field changed from `local` to `docker`. The `host` field changed from the EC2 hostname to the container ID. The Node version and the tasks result are identical, because the runtime is now pinned inside the image.

✏️ **Reflect:** Compare the `host` field in `GET /` between Part A and Part B. Why are they different? What does this tell you about how a container relates to the machine it runs on?

10. Demonstrate reproducibility

```
docker stop myapp && docker rm myapp
docker ps -a
docker run -d -p 3000:3000 --name myapp lab6-app
curl http://localhost:3000/tasks
```

The result is identical to before. The container was destroyed and recreated from the same image, and nothing changed.

✏️ **Reflect:** You destroyed the container and started a fresh one, yet the output was identical. Why? What does this say about the role of the image versus the container?

11. Explore inside the container

Open an interactive shell inside the running container:

```
docker exec -it myapp sh
```

You are now inside the container's isolated filesystem. Run a few commands to explore:

```
# Where are we?
pwd

# Files copied in by the Dockerfile
ls /app

# Node version inside the container — always 22, regardless of nvm on the
host
node --version

# Hostname matches the container ID shown in GET / and docker ps
hostname

# Exit the container shell
exit
```

💡 **Note:** `node --version` reports v22 inside the container regardless of what `nvm use` is set to on the host. The hostname matches the container ID you saw in `docker ps` and in the `host` field of `GET /`. The container carries its own isolated environment.

⚠️ **Warning:** This image is based on Alpine Linux, which does not include `bash`. Use `sh` instead. Running `docker exec -it myapp bash` will return `not found`.

Cleanup

🚨 **Caution:** EC2 instances accrue charges while running. Always terminate your instance after the lab.

On the EC2 instance (before disconnecting):

```
docker rm -f myapp
docker rmi lab6-app
```

In the AWS Console:

1. EC2 → Instances → select your instance
2. **Instance State** → **Terminate instance** → confirm

🔧 **Challenge:** The `mode` field in `GET /` is set using an environment variable (`ENV MODE=docker` in the Dockerfile). Can you override it at runtime without changing the Dockerfile? Try running the container with `-e MODE=test` and check the response from `GET /`.